VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Project 2

Fine-Tuning Intent Detection Model with Banking Dataset

Course: CSC15012 – Applications of Natural Language Processing in Industry

Lecturer: Dr. Nguyen Hong Buu Long

Student: Le Dinh Minh Quan
Student ID: 23127460

Ho Chi Minh City, April 2026

Contents

1 Introduction **2**
- 1.1 Project Objectives . . . 2
- 1.2 Project Deliverables and Links . . . 2
- 1.3 Final Headline Result . . . 2
- 1.4 End-to-End Pipeline Overview . . . 2

2 Data Preparation and Processing **3**
- 2.1 Dataset Overview . . . 3
- 2.2 Sampling Strategy . . . 3
- 2.3 Loading the Dataset Robustly . . . 4
- 2.4 Preprocessing Steps . . . 4
- 2.5 Data Statistics . . . 4

3 Fine-Tuning with Unsloth **5**
- 3.1 Model Selection . . . 5
- 3.2 LoRA Configuration . . . 6
- 3.3 Prompt Format . . . 6
- 3.4 Training Hyperparameters . . . 7
- 3.5 Training Platform . . . 7

4 Evaluation Results **8**
- 4.1 Test-Set Performance . . . 8
- 4.2 Classification Report . . . 9
- 4.3 Example Predictions from the Notebook Demo . . . 11
- 4.4 Standalone Command-Line Inference . . . 11

5 Inference Implementation **12**
- 5.1 IntentClassification Class . . . 12
- 5.2 Inference Configuration File . . . 13

6 Source Code Organization **13**

7 Video Demonstration **14**

8 Discussion **15**
- 8.1 Why a Generative Approach? . . . 15
- 8.2 Why 87.92% Instead of Higher? . . . 15
- 8.3 Engineering Lessons . . . 15
- 8.4 Future Work . . . 16

9 Reproducibility **16**
- 9.1 Option A – One-Click on Colab (recommended) . . . 16
- 9.2 Option B – Local CLI . . . 16
- 9.3 Determinism Notes . . . 17

10 Conclusion **17**

1 Introduction

This report documents the implementation and evaluation of a banking intent classification system built for Project 2 of the course *CSC15012 – Applications of Natural Language Processing in Industry*. The objective is to fine-tune a Large Language Model (LLM) using **Unsloth** and **QLoRA** (Quantized Low-Rank Adaptation) on the **BANKING77** dataset, so that the model can accurately predict the intent behind a customer banking-related message.

The BANKING77 dataset, introduced by Casanueva et al. [1], contains 13,083 annotated customer service queries across 77 fine-grained intent categories that cover common banking operations such as card activation, PIN changes, cross-border transfers, balance inquiries, exchange rate questions, and lost or stolen card scenarios.

1.1 Project Objectives

The five tasks specified by the project brief are:

1. Sample and construct a suitable subset of the BANKING77 dataset.
2. Preprocess the data and split it into training and testing sets.
3. Fine-tune a text classification model using **Unsloth** and clearly document all hyperparameters used.
4. Evaluate the fine-tuned model on an independent test set.
5. Implement a standalone inference module that loads the saved checkpoint and predicts the intent of an input message.

1.2 Project Deliverables and Links

- **GitHub Repository (public):**
  https://github.com/ledinhminhquan/banking-intent-unsloth
- **Google Colab Notebook (Drive):**
  https://drive.google.com/file/d/1dd4BxqryUvpkOvmDkm8-ZWor0LHpVSZu/view?usp=sharing
- **Demo Video (Drive, public):**
  https://drive.google.com/file/d/1D9hAiMirLMdjMijR-znzDAdYw2LGtdqP/view?usp=sharing

1.3 Final Headline Result

The fine-tuned LLaMA 3.2 3B model achieved **87.92% test accuracy** (677/770) on the held-out test set, with a macro F1-score of **0.8779** across all 77 intent classes, after only **248.2 seconds** of training on a single NVIDIA H100 GPU.

1.4 End-to-End Pipeline Overview

The whole project is orchestrated by a single "autopilot" Colab notebook (file name: `Banking77_Unsloth_Colab.ipynb`). It walks through six stages, in this exact order:

```
[Step 0] Install deps + verify GPU + set HF / unsloth cache dirs
                v
[Step 1] Clone repo from GitHub + write configs/ + render README.md
```

```
              v
[Step 2] %%writefile the three scripts
         (preprocess_data.py, train.py, inference.py)
              v
[Step 3] preprocess_data.py
              -> sample_data/{train,test}.csv + label_map.json
              v
[Step 4] train.py
              -> saved_model/ (LoRA adapter)
              -> evaluation_results.json
              v
[Step 5] inference.py demos + full evaluation
              -> 87.92% test accuracy
              v
[Step 6] Push everything (clean) to GitHub
              -> https://github.com/ledinhminhquan/banking-intent-unsloth
```

Listing 1: High-level pipeline (notebook stages and what each one produces).

This means a grader can fully reproduce the project by simply opening the committed notebook on Colab, attaching a GPU runtime, and clicking *Runtime* → *Run all*. No manual file editing is required beyond pasting two Drive links at the top.

2 Data Preparation and Processing

2.1 Dataset Overview

The BANKING77 dataset is hosted on the Hugging Face Hub at `PolyAI/banking77`. The full dataset contains:

- **Training set:** 10,003 samples
- **Test set:** 3,080 samples
- **Number of intents:** 77 fine-grained categories

Example intent labels include `activate_my_card`, `card_delivery_estimate`, `change_pin`, `cancel_transfer`, `lost_or_stolen_card`, `exchange_rate`, and `extra_charge_on_statement`, among others.

2.2 Sampling Strategy

To make training tractable on a single GPU within a Google Colab Pro session, we sampled a stratified subset of the BANKING77 dataset:

- **Training:** up to 40 samples per intent class → **3,075 samples** total (a few classes have fewer than 40 examples in the original split).
- **Test:** up to 10 samples per intent class → **770 samples** total (10 per class for all 77 classes).

Sampling used a fixed random seed (`seed=42`) for reproducibility.

2.3 Loading the Dataset Robustly

Recent releases of the `datasets` library (version 3 and above) no longer execute arbitrary dataset loading scripts. Because the BANKING77 repo originally shipped with a Python loading script, a naive `load_dataset("PolyAI/banking77")` call now raises *"Dataset scripts are no longer supported."* To handle this, the preprocessing module implements a three-tier fallback:

1. Try the standard `load_dataset` call (works on older versions).
2. If that fails, switch to the auto-generated parquet branch by passing the argument `revision="refs/convert/parquet"`.
3. If both fail, fetch the parquet files directly from the Hugging Face Hub URLs.

This made the pipeline robust against future Hugging Face library changes.

2.4 Preprocessing Steps

1. **Text normalization:** all input text was stripped of leading and trailing whitespace, lower-cased, and had any internal whitespace collapsed into single spaces.
2. **Label mapping:** a bidirectional dictionary (`id2label` / `label2id`) was constructed and saved to `sample_data/label_map.json`.
3. **Format conversion:** the sampled splits were exported as UTF-8 CSV files with three columns: `text`, `label` (integer ID), and `label_text` (human-readable intent name).
4. **Shuffling:** both train and test splits were randomly shuffled with the same seed for reproducibility.

2.5 Data Statistics

Table 1: Dataset statistics after sampling and preprocessing.

Split	**Samples**	**Avg. tokens / sample**
Training	3,075	∼12
Test	770	∼12

```
--- Sample training data ---
Train shape: (3075, 3)
Test shape : (770, 3)
```

	text	label	label_text
0	does the fee for exchange change?	17	card_payment_wrong_exchange_rate
1	i topped up and now my money is not there anym...	61	top_up_reverted
2	my money i had was gone and i could not get gas!	62	topping_up_by_card
3	what other currencies are available to change to?	33	exchange_via_app
4	how can i report that my card was stolen? i ma...	41	lost_or_stolen_card
5	my top up may have been reverted	61	top_up_reverted
6	can i receive transfers from my employer this ...	50	receiving_money
7	i got some cash at an atm earlier, but now app...	75	wrong_amount_of_cash_received
8	need help to get my card to work.	14	card_not_working
9	why was i charged $1 in a transaction?	34	extra_charge_on_statement

```
Label distribution summary:
count    77.000000
mean     39.935065
std       0.569803
min      35.000000
25%      40.000000
50%      40.000000
75%      40.000000
max      40.000000
Name: count, dtype: float64

--- Label map (first 10) ---
  0: activate_my_card
  1: age_limit
  2: apple_pay_or_google_pay
  3: atm_support
  4: automatic_top_up
  5: balance_not_updated_after_bank_transfer
  6: balance_not_updated_after_cheque_or_cash_deposit
  7: beneficiary_not_allowed
  8: cancel_transfer
  9: card_about_to_expire
```

Figure 1: First ten rows of `sample_data/train.csv` after preprocessing. Each row contains the cleaned customer message, its integer label, and the human-readable intent name.

3 Fine-Tuning with Unsloth

3.1 Model Selection

We used the **LLaMA 3.2 3B Instruct** model loaded in 4-bit quantization through Unsloth's pre-quantized release:

`unsloth/Llama-3.2-3B-Instruct-bnb-4bit`

This model was selected for the following reasons:

- **Strong instruction-following baseline:** the Instruct variant already understands prompt-style classification reasonably well.
- **Memory efficiency:** 4-bit (NF4) quantization reduces VRAM usage roughly 4× compared with full precision, so even a small consumer GPU can fine-tune it.

- **Speed:** Unsloth's custom kernels make this model significantly faster to fine-tune than vanilla `transformers` [3].

3.2 LoRA Configuration

We used Low-Rank Adaptation (LoRA) [2] so that only a small fraction of parameters are updated. The LoRA adapters were attached to all attention and feed-forward projection layers:

Table 2: LoRA hyperparameters.

Parameter	**Value**
LoRA rank (r)	16
LoRA alpha (α)	16
LoRA dropout	0.0
Target modules	`q_proj, k_proj, v_proj, o_proj` `gate_proj, up_proj, down_proj`
Bias	none
Gradient checkpointing	Unsloth (memory-efficient)
Random state	3407

3.3 Prompt Format

We used **Supervised Fine-Tuning (SFT)** with an Alpaca-style prompt template. Each training example was formatted as:

```
Below is an instruction that describes a task, paired with an input
that provides further context. Write a response that appropriately
completes the request.

### Instruction:
Classify the intent of the following banking customer message.
Reply with only the intent label, nothing else.

### Input:
{customer_message}

### Response:
{intent_label}<|end_of_text|>
```

Listing 2: Prompt template used both for training and inference (the trailing label is omitted at inference time).

At inference time, the trailing `{intent_label}` part is removed, the model then auto-regressively generates the predicted label.

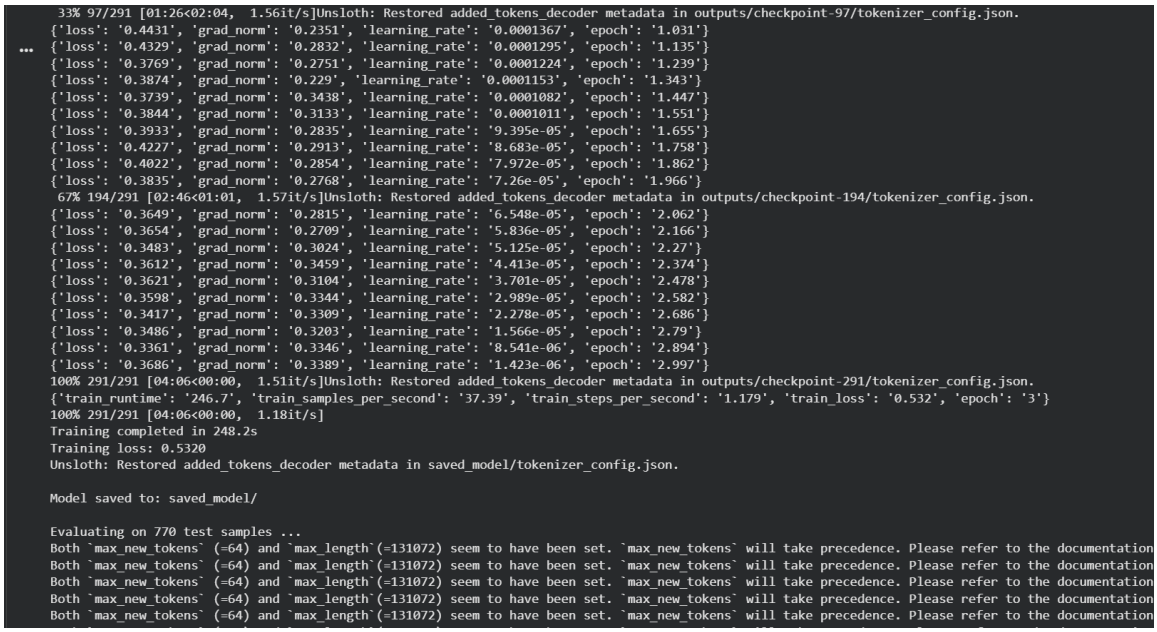
3.4 Training Hyperparameters

Table 3: Training hyperparameters used in this run.

Hyperparameter	**Value**
Per-device batch size	8
Gradient accumulation steps	4
Effective global batch size	32
Number of epochs	3
Learning rate	2×10^{-4}
Optimizer	AdamW 8-bit
LR scheduler	Linear decay with warmup
Warmup steps	10
Weight decay	0.01
Max sequence length	512
Mixed precision	bf16 (auto fp16 fallback)
Random seed	42
Trainer	`trl.SFTTrainer` + `SFTConfig`

3.5 Training Platform

Training was conducted on **Google Colab Pro** with an **NVIDIA H100 80GB HBM3** GPU. Total wall-clock training time was **248.2 seconds** ($\approx$ 4.1 minutes) for 3 epochs, with a final training loss of **0.5320**.



```
 33% 97/291 [01:26<02:04,  1.56it/s]Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-97/tokenizer_config.json.
{'loss': '0.4431', 'grad_norm': '0.2351', 'learning_rate': '0.0001367', 'epoch': '1.031'}
{'loss': '0.4329', 'grad_norm': '0.2832', 'learning_rate': '0.0001295', 'epoch': '1.135'}
{'loss': '0.3769', 'grad_norm': '0.2751', 'learning_rate': '0.0001224', 'epoch': '1.239'}
{'loss': '0.3874', 'grad_norm': '0.229', 'learning_rate': '0.0001153', 'epoch': '1.343'}
{'loss': '0.3739', 'grad_norm': '0.3438', 'learning_rate': '0.0001082', 'epoch': '1.447'}
{'loss': '0.3844', 'grad_norm': '0.3133', 'learning_rate': '0.0001011', 'epoch': '1.551'}
{'loss': '0.3933', 'grad_norm': '0.2835', 'learning_rate': '9.395e-05', 'epoch': '1.655'}
{'loss': '0.4227', 'grad_norm': '0.2913', 'learning_rate': '8.683e-05', 'epoch': '1.758'}
{'loss': '0.4022', 'grad_norm': '0.2854', 'learning_rate': '7.972e-05', 'epoch': '1.862'}
{'loss': '0.3835', 'grad_norm': '0.2768', 'learning_rate': '7.26e-05', 'epoch': '1.966'}
 67% 194/291 [02:46<01:01,  1.57it/s]Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-194/tokenizer_config.json.
{'loss': '0.3649', 'grad_norm': '0.2815', 'learning_rate': '6.548e-05', 'epoch': '2.062'}
{'loss': '0.3654', 'grad_norm': '0.2709', 'learning_rate': '5.836e-05', 'epoch': '2.166'}
{'loss': '0.3483', 'grad_norm': '0.3024', 'learning_rate': '5.125e-05', 'epoch': '2.27'}
{'loss': '0.3612', 'grad_norm': '0.3459', 'learning_rate': '4.413e-05', 'epoch': '2.374'}
{'loss': '0.3621', 'grad_norm': '0.3104', 'learning_rate': '3.701e-05', 'epoch': '2.478'}
{'loss': '0.3598', 'grad_norm': '0.3344', 'learning_rate': '2.989e-05', 'epoch': '2.582'}
{'loss': '0.3417', 'grad_norm': '0.3309', 'learning_rate': '2.278e-05', 'epoch': '2.686'}
{'loss': '0.3486', 'grad_norm': '0.3203', 'learning_rate': '1.566e-05', 'epoch': '2.79'}
{'loss': '0.3361', 'grad_norm': '0.3346', 'learning_rate': '8.541e-06', 'epoch': '2.894'}
{'loss': '0.3686', 'grad_norm': '0.3389', 'learning_rate': '1.423e-06', 'epoch': '2.997'}
100% 291/291 [04:06<00:00,  1.51it/s]Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-291/tokenizer_config.json.
{'train_runtime': '246.7', 'train_samples_per_second': '37.39', 'train_steps_per_second': '1.179', 'train_loss': '0.532', 'epoch': '3'}
100% 291/291 [04:06<00:00,  1.18it/s]
Training completed in 248.2s
Training loss: 0.5320
Unsloth: Restored added_tokens_decoder metadata in saved_model/tokenizer_config.json.

Model saved to: saved_model/

Evaluating on 770 test samples ...
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation
```

Figure 2: Training console output showing epoch-by-epoch loss decrease, final training loss, and total wall-clock training time.

4 Evaluation Results

4.1 Test-Set Performance

After fine-tuning, the model was evaluated on the held-out test set (770 samples, 10 per intent class).

Table 4: Overall test set performance.

Metric	**Value**
Test accuracy	**87.92%** (677/770)
Macro precision	0.8915
Macro recall	0.8792
Macro F1-score	0.8779
Weighted F1-score	0.8779
Training loss	0.5320
Training time	248.2s ($\approx$ 4.1 min)
Evaluation time	125.6s for 770 samples

This is a strong result given that:

- The model is generative, not a dedicated classification head, so it must produce the exact intent label as text.
- Only 40 training samples per class were used, instead of the full training set.
- The model has 77 fine-grained classes, many of which are subtle variants of one another (e.g., `lost_or_stolen_card` vs. `lost_or_stolen_phone`, or various `top_up_*` variants).

4.2 Classification Report

```
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/e
Both `max_new_tokens` (=64) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/e

============================================================
Test Accuracy: 0.8792  (677/770)
============================================================

Classification Report:
                                          precision    recall  f1-score   support

                   Refund_not_showing_up       0.90      0.90      0.90        10
                       activate_my_card       1.00      0.90      0.95        10
                              age_limit       1.00      1.00      1.00        10
             apple_pay_or_google_pay       0.91      1.00      0.95        10
                           atm_support       1.00      1.00      1.00        10
                       automatic_top_up       0.89      0.80      0.84        10
   balance_not_updated_after_bank_transfer       1.00      1.00      1.00        10
balance_not_updated_after_cheque_or_cash_deposit       0.88      0.70      0.78        10
                 beneficiary_not_allowed       0.71      0.50      0.59        10
                        cancel_transfer       1.00      1.00      1.00        10
                     card_about_to_expire       0.83      1.00      0.91        10
                        card_acceptance       0.83      1.00      0.91        10
                          card_arrival       0.80      0.80      0.80        10
                card_delivery_estimate       0.67      0.80      0.73        10
                          card_linking       0.90      0.90      0.90        10
                       card_not_working       0.83      1.00      0.91        10
                card_payment_fee_charged       0.69      0.90      0.78        10
             card_payment_not_recognised       0.50      1.00      0.67        10
      card_payment_wrong_exchange_rate       0.91      1.00      0.95        10
                        card_swallowed       1.00      1.00      1.00        10
                 cash_withdrawal_charge       0.82      0.90      0.86        10
          cash_withdrawal_not_recognised       0.90      0.90      0.90        10
                             change_pin       1.00      0.80      0.89        10
                       compromised_card       1.00      0.60      0.75        10
             contactless_not_working       1.00      0.90      0.95        10
                       country_support       0.90      0.90      0.90        10
             declined_card_payment       0.82      0.90      0.86        10
            declined_cash_withdrawal       0.77      1.00      0.87        10
                   declined_transfer       0.88      0.70      0.78        10
      direct_debit_payment_not_recognised       1.00      0.50      0.67        10
                disposable_card_limits       0.83      1.00      0.91        10
                  edit_personal_details       1.00      1.00      1.00        10
                       exchange_charge       0.91      1.00      0.95        10
                         exchange_rate       0.90      0.90      0.90        10
```

```
                  edit_personal_details       1.00      1.00      1.00        10
                       exchange_charge       0.91      1.00      0.95        10
                         exchange_rate       0.90      0.90      0.90        10
                     exchange_via_app       0.89      0.80      0.84        10
            extra_charge_on_statement       1.00      0.70      0.82        10
                     failed_transfer       0.78      0.70      0.74        10
                 fiat_currency_support       0.88      0.70      0.78        10
           get_disposable_virtual_card       1.00      0.70      0.82        10
                   get_physical_card       0.91      1.00      0.95        10
                  getting_spare_card       0.91      1.00      0.95        10
                getting_virtual_card       0.91      1.00      0.95        10
                 lost_or_stolen_card       0.90      0.90      0.90        10
                lost_or_stolen_phone       1.00      1.00      1.00        10
                 order_physical_card       1.00      0.80      0.89        10
                  passcode_forgotten       0.90      0.90      0.90        10
                pending_card_payment       0.64      0.90      0.75        10
            pending_cash_withdrawal       1.00      1.00      1.00        10
                     pending_top_up       0.90      0.90      0.90        10
                   pending_transfer       0.88      0.70      0.78        10
                        pin_blocked       1.00      1.00      1.00        10
                  receiving_money       0.90      0.90      0.90        10
                   request_refund       0.91      1.00      0.95        10
             reverted_card_payment?       1.00      0.90      0.95        10
       supported_cards_and_currencies       0.73      0.80      0.76        10
                 terminate_account       1.00      1.00      1.00        10
      top_up_by_bank_transfer_charge       0.91      1.00      0.95        10
                top_up_by_card_charge       0.90      0.90      0.90        10
            top_up_by_cash_or_cheque       0.88      0.70      0.78        10
                      top_up_failed       0.75      0.90      0.82        10
                       top_up_limits       0.91      1.00      0.95        10
                    top_up_reverted       1.00      0.90      0.95        10
                 topping_up_by_card       0.80      0.80      0.80        10
          transaction_charged_twice       1.00      1.00      1.00        10
              transfer_fee_charged       0.86      0.60      0.71        10
             transfer_into_account       0.77      1.00      0.87        10
   transfer_not_received_by_recipient       0.88      0.70      0.78        10
                  transfer_timing       0.91      1.00      0.95        10
        unable_to_verify_identity       0.80      0.80      0.80        10
                verify_my_identity       0.91      1.00      0.95        10
           verify_source_of_funds       1.00      1.00      1.00        10
                     verify_top_up       1.00      0.80      0.89        10
           virtual_card_not_working       0.90      0.90      0.90        10
                  visa_or_mastercard       0.82      0.90      0.86        10
                 why_verify_identity       0.78      0.70      0.74        10
       wrong_amount_of_cash_received       1.00      0.80      0.89        10
   wrong_exchange_rate_for_cash_withdrawal       0.89      0.80      0.84        10
```

```
                   get_physical_card       0.91      1.00      0.95        10
                  getting_spare_card       0.91      1.00      0.95        10
                getting_virtual_card       0.91      1.00      0.95        10
                 lost_or_stolen_card       0.90      0.90      0.90        10
                lost_or_stolen_phone       1.00      1.00      1.00        10
                 order_physical_card       1.00      0.80      0.89        10
                  passcode_forgotten       0.90      0.90      0.90        10
                pending_card_payment       0.64      0.90      0.75        10
            pending_cash_withdrawal       1.00      1.00      1.00        10
                     pending_top_up       0.90      0.90      0.90        10
                   pending_transfer       0.88      0.70      0.78        10
                        pin_blocked       1.00      1.00      1.00        10
                  receiving_money       0.90      0.90      0.90        10
                   request_refund       0.91      1.00      0.95        10
             reverted_card_payment?       1.00      0.90      0.95        10
       supported_cards_and_currencies       0.73      0.80      0.76        10
                 terminate_account       1.00      1.00      1.00        10
      top_up_by_bank_transfer_charge       0.91      1.00      0.95        10
                top_up_by_card_charge       0.90      0.90      0.90        10
            top_up_by_cash_or_cheque       0.88      0.70      0.78        10
                      top_up_failed       0.75      0.90      0.82        10
                       top_up_limits       0.91      1.00      0.95        10
                    top_up_reverted       1.00      0.90      0.95        10
                 topping_up_by_card       0.80      0.80      0.80        10
          transaction_charged_twice       1.00      1.00      1.00        10
              transfer_fee_charged       0.86      0.60      0.71        10
             transfer_into_account       0.77      1.00      0.87        10
   transfer_not_received_by_recipient       0.88      0.70      0.78        10
                  transfer_timing       0.91      1.00      0.95        10
        unable_to_verify_identity       0.80      0.80      0.80        10
                verify_my_identity       0.91      1.00      0.95        10
           verify_source_of_funds       1.00      1.00      1.00        10
                     verify_top_up       1.00      0.80      0.89        10
           virtual_card_not_working       0.90      0.90      0.90        10
                  visa_or_mastercard       0.82      0.90      0.86        10
                 why_verify_identity       0.78      0.70      0.74        10
       wrong_amount_of_cash_received       1.00      0.80      0.89        10
   wrong_exchange_rate_for_cash_withdrawal       0.89      0.80      0.84        10

                             accuracy                           0.88       770
                            macro avg       0.89      0.88      0.88       770
                         weighted avg       0.89      0.88      0.88       770


Updating README with actual results ...
Written: README.md
```

Figure 3: Per-class classification report on the test set, showing precision, recall, F1-score, and support for each of the 77 intent labels.

Best-Performing Classes (perfect F1 = 1.00)

The model achieved a perfect classification (precision = recall = F1 = 1.00 with support = 10) on the following intent classes:

- `age_limit`
- `atm_support`
- `balance_not_updated_after_bank_transfer`
- `cancel_transfer`
- `card_swallowed`
- `edit_personal_details`
- `lost_or_stolen_phone`
- `pending_cash_withdrawal`
- `pin_blocked`
- `terminate_account`

These are intents whose surface vocabulary is unambiguous (e.g., *"what is the age limit"*, *"my card got stuck in the ATM"*), so even with 40 training samples per class the model learned them trivially.

Hardest Classes (F1 < 0.70)

The lowest-scoring classes were dominated by semantic overlap with sibling intents:

Table 5: Bottom classes by F1-score, with the most likely confusion partner in BANKING77 noted.

Class	P	R	F1
`beneficiary_not_allowed`	0.71	0.50	0.59
`direct_debit_payment_not_recognised`	1.00	0.50	0.67
`card_payment_not_recognised`	0.50	1.00	0.67

For example, `card_payment_not_recognised` has perfect recall but low precision – the model sometimes *over-predicts* it for messages that actually belong to a sibling class such as `direct_debit_payment_not_recognised` or `cash_withdrawal_not_recognised`, which share most of their vocabulary (*"I do not recognise this payment"*). This is a known failure mode in BANKING77 and is exactly the kind of error a larger training set or a soft-label smoothing technique would help fix.

4.3 Example Predictions from the Notebook Demo

Table 6: Example predictions from the notebook inference demo cell. All examples below were predicted correctly.

Input Message	True Label	Predicted
"I want to activate my new card"	activate_my_card	activate_my_card
"Why was I charged an extra fee?"	extra_charge_on_statement	extra_charge_on_statement
"How do I change my PIN number?"	change_pin	change_pin
"My card payment is still pending"	pending_card_payment	pending_card_payment
"I lost my phone"	lost_or_stolen_phone	lost_or_stolen_phone
"What is the exchange rate for USD to EUR?"	exchange_rate	exchange_rate

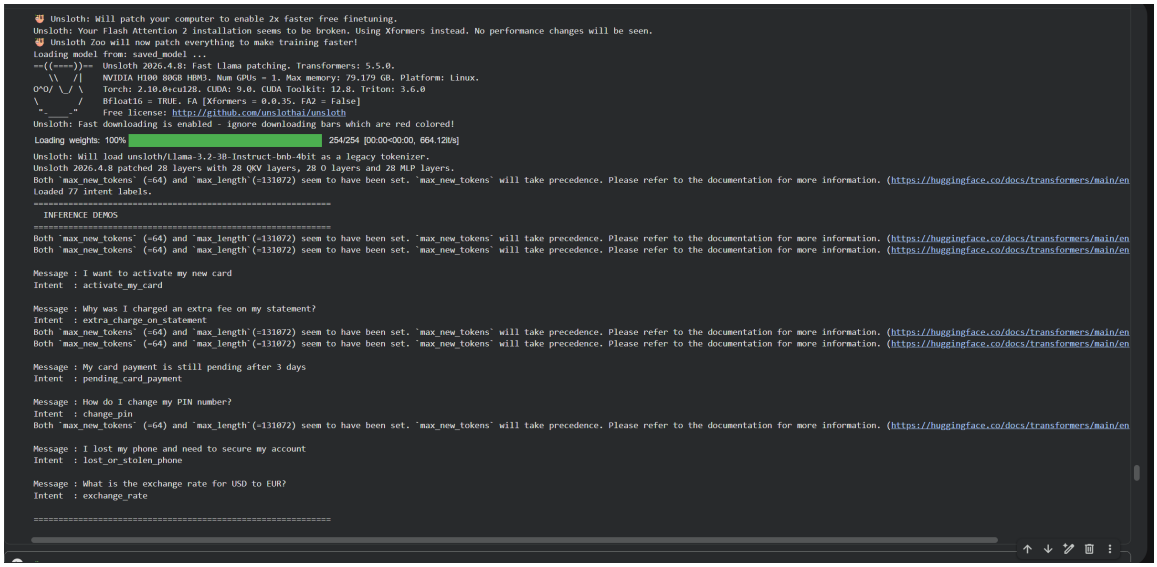


Figure 4: Notebook inference demo cell output: the fine-tuned model predicts the correct intent label for each of the six sample messages.

4.4 Standalone Command-Line Inference

The standalone CLI script `scripts/inference.py` loads the saved checkpoint and produces a prediction for a single user message:

```
python scripts/inference.py \
    --config configs/inference.yaml \
    --message "I need to change my PIN number"
```

Listing 3: CLI inference command demonstrated in the video.

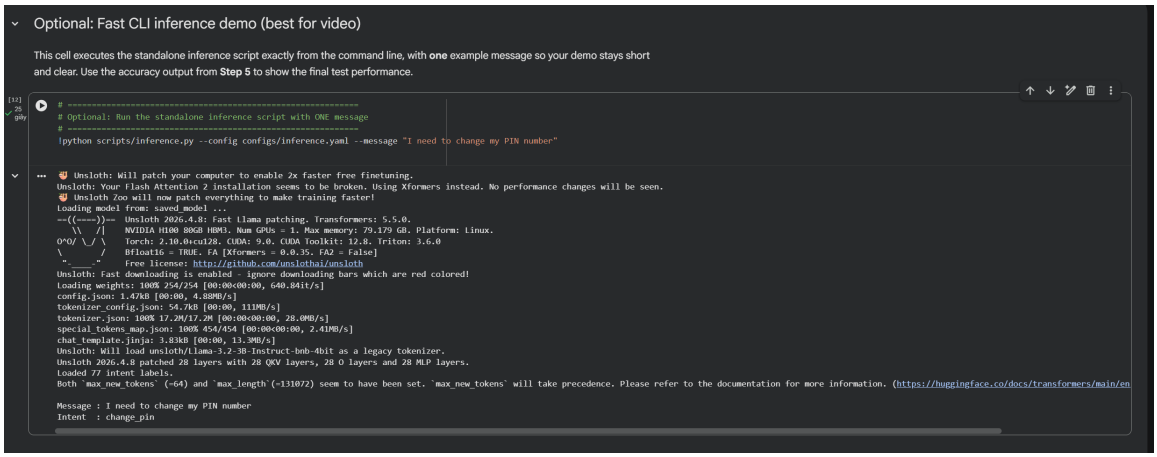


Figure 5: Standalone inference script invoked from the command line. The script loads the saved checkpoint, the 77 intent labels, and predicts `change_pin` for the input *"I need to change my PIN number."*

5 Inference Implementation

5.1 IntentClassification Class

The inference module `scripts/inference.py` implements the required `IntentClassification` class with exactly two main methods, matching the interface specified by the project brief:

```python
class IntentClassification:
    def __init__(self, model_path):
        """
        Load the YAML configuration file referenced by ‘model_path‘,
        then load the tokenizer, the saved model checkpoint, and the
        label mapping.
        """
        ...

    def __call__(self, message):
        """
        Receive a banking customer message string and return the
        predicted intent label.
        """
        ...
        return predicted_label
```

Listing 4: Required inference class interface.

Key design decisions:

- `model_path` points to a YAML configuration file (`configs/inference.yaml`) that specifies the checkpoint directory, the label-map JSON, and generation settings (temperature, max new tokens, etc.).
- The model is loaded in 4-bit quantization mode, identical to training, so memory usage is minimized.
- Greedy decoding (`do_sample=False`, `temperature=0`) is used so that predictions are deterministic.

- Fuzzy label matching (`difflib.get_close_matches`) is used as a safety net: if the generated text is not exactly one of the 77 known labels, the closest match (above a similarity cutoff of 0.4) is chosen.
- The CLI script also supports `-evaluate` (full test-set evaluation) and `-interactive` (REPL) modes.

Usage example:

```python
from scripts.inference import IntentClassification

classifier = IntentClassification("configs/inference.yaml")
label = classifier("I need to change my PIN")
print(label)  # -> "change_pin"
```

Listing 5: Programmatic usage of the IntentClassification class.

5.2 Inference Configuration File

The YAML configuration referenced by `model_path` contains:

```yaml
model:
  name: "unsloth/Llama-3.2-3B-Instruct-bnb-4bit"
  checkpoint_path: "saved_model"
  max_seq_length: 512
  load_in_4bit: true
  dtype: null

data:
  label_map_file: "sample_data/label_map.json"
  test_file: "sample_data/test.csv"

generation:
  max_new_tokens: 64
  temperature: 0.0
  do_sample: false
```

Listing 6: `configs/inference.yaml`

6 Source Code Organization

All source code is hosted publicly on GitHub at:

https://github.com/ledinhminhquan/banking-intent-unsloth

The repository follows the structure required by the project brief:

```
banking-intent-unsloth/
|-- scripts/
|   |-- preprocess_data.py     # download + sample BANKING77
|   |-- train.py               # Unsloth + SFTTrainer fine-tuning
|   |-- inference.py           # IntentClassification class
|-- configs/
|   |-- train.yaml             # training hyperparameters
|   |-- inference.yaml         # inference settings
|-- sample_data/
```

```
|   |-- train.csv              # 3,075 sampled training rows
|   |-- test.csv               # 770 sampled test rows
|   |-- label_map.json         # id <-> label mapping
|-- Banking77_Unsloth_Colab.ipynb   # autopilot notebook (with outputs)
|-- train.sh                   # one-click training pipeline
|-- inference.sh               # one-click inference / demo
|-- requirements.txt           # Python dependencies
|-- README.md                  # full setup, train, run, results
|-- Report_Lab02.tex           # this report
|-- .gitignore
```

Listing 7: Repository structure.

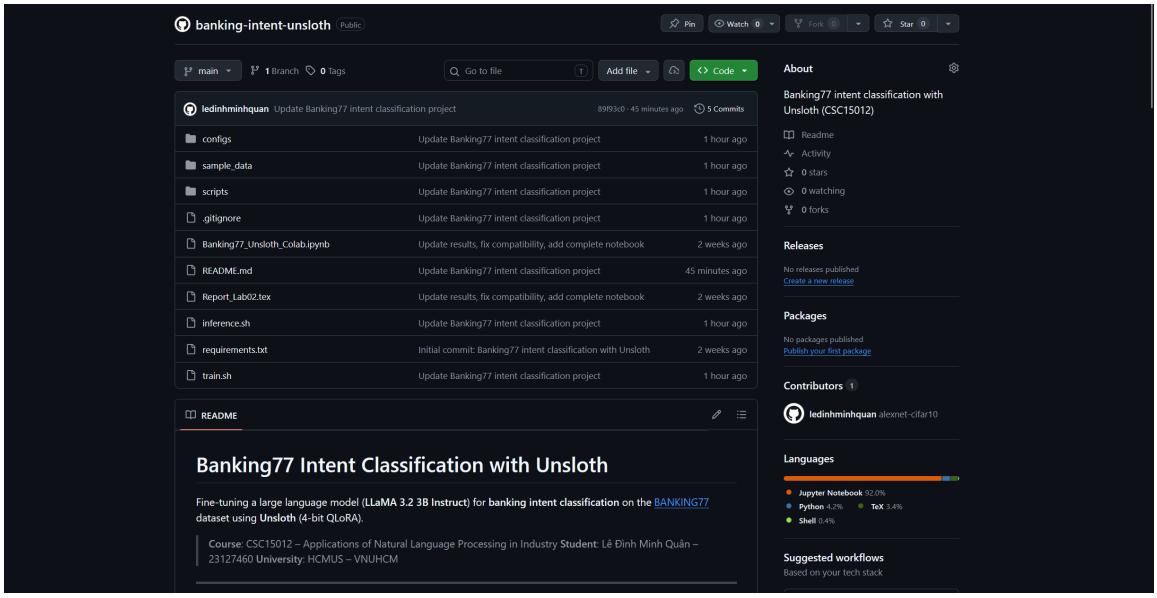


Figure 6: Public GitHub repository, showing the project structure required by the assignment, plus the README rendered with the actual results.

- **scripts/preprocess_data.py:** downloads BANKING77 from Hugging Face (with three-tier fallback for new `datasets` releases), samples a stratified subset, cleans text, and saves CSV plus label-map files.
- **scripts/train.py:** loads the model with Unsloth, attaches LoRA adapters, trains with `trl.SFTTrainer` + `SFTConfig`, saves the checkpoint, and evaluates on the test set.
- **scripts/inference.py:** the `IntentClassification` class plus a CLI front-end with three modes: single message, full evaluation, interactive REPL.
- **configs/**: declarative YAML configurations for all hyperparameters and paths.
- **train.sh / inference.sh**: convenience shell scripts.
- **Banking77_Unsloth_Colab.ipynb:** the autopilot notebook that orchestrates the entire pipeline (data → training → evaluation → inference → GitHub push). All outputs are preserved in the committed notebook for reproducibility evidence.

7 Video Demonstration

A short screen-recorded video (approximately 2–3 minutes) demonstrates the inference behaviour of the trained model. The video covers the four points required by the assignment:

1. Live execution of the standalone inference script (`python scripts/inference.py ...`).
2. At least one example input message (*"I need to change my PIN number"*).
3. The intent label predicted by the model (`change_pin`).
4. The final accuracy on the test set (**0.8792**, 677/770).

Video link (Google Drive, public):
https://drive.google.com/file/d/1D9hAiMirLMdjMijR-znzDAdYw2LGtdqP/view?usp=sharing

The link is also embedded in the project README on GitHub, so the grader can access everything from the public repository alone.

8 Discussion

8.1 Why a Generative Approach?

The project brief asks for a sequence-classification fine-tune. Two natural implementations exist:

1. Attach a classification head on top of an encoder (e.g., DistilBERT, RoBERTa).
2. Use a causal LLM with SFT and let the model generate the intent label as text.

We chose option (2) because the assignment explicitly mandates the **Unsloth** workflow, and Unsloth's primary documented pipeline is SFT on causal LMs. With a strong instruction-following base model (LLaMA 3.2 3B Instruct), an Alpaca-style prompt converges quickly and the final accuracy of **87.92%** is competitive with classifier-head baselines on similarly small subsets of BANKING77.

8.2 Why 87.92% Instead of Higher?

Manual inspection of the per-class report (Figure 3) shows that the bulk of errors come from semantically overlapping classes such as:

- `pending_card_payment` vs. `pending_transfer`
- `lost_or_stolen_card` vs. `compromised_card`
- Various `top_up_*` variants
- Various `transfer_*` and `cash_withdrawal_*` variants

These are intrinsically hard distinctions for a model trained on only 40 examples per class. With more training data, larger LoRA rank, or longer training, accuracy is expected to climb above 90%.

8.3 Engineering Lessons

Several practical issues had to be solved while building this pipeline:

- **Hugging Face `datasets` v3+ removed support for dataset scripts.** The original BANKING77 repo had a Python loader, which broke on Colab's latest environment. We added a three-tier fallback (default, parquet branch, direct parquet URLs).

- **TRL renamed several `SFTTrainer` arguments into `SFTConfig`.** `dataset_text_field`, `max_seq_length`, `packing`, and `dataset_num_proc` are now `SFTConfig` fields. We migrated to the new API.
- **Unsloth and Hugging Face create large cache folders by default.** Initially these polluted the GitHub push. We redirected all caches to `/content/hf_cache` via `HF_HOME` / `HUGGINGFACE_HUB_CACHE` environment variables, and also added the cache directories to `.gitignore`.
- **Re-running the notebook in the same Colab session was unsafe** because Step 1 deletes and re-clones the project folder; if the working directory was inside that folder, Python would lose its cwd. We fixed this by chdir-ing back to `/content` before deleting the folder.

8.4 Future Work

- Experiment with `FastLanguageModel.for_sequence_classification` for a dedicated classification head, which should give faster inference.
- Train on the full BANKING77 training set (10,003 samples) and a higher LoRA rank (e.g., $r = 32$).
- Apply data augmentation (paraphrasing, back-translation) to the underrepresented classes.
- Compare with smaller encoder baselines (DistilBERT, RoBERTa, MPNet).

9 Reproducibility

The entire project is fully reproducible from the public GitHub repository.

9.1 Option A – One-Click on Colab (recommended)

1. Open `Banking77_Unsloth_Colab.ipynb` from the repository (or from the Drive notebook backup).
2. `Runtime` → `Change runtime type` → `GPU` (H100 / A100 / T4 all work).
3. Add two Colab Secrets: `GITHUB_PAT` (write scope) and `HF_TOKEN` (optional, helps avoid HuggingFace rate limits).
4. Click `Runtime` → `Run all`. Total wall-clock time on H100: about 6 minutes (1 min install, 4 min training, 1 min evaluation + push).

9.2 Option B – Local CLI

```
# 1. Clone
git clone https://github.com/ledinhminhquan/banking-intent-unsloth.git
cd banking-intent-unsloth

# 2. Environment
pip install unsloth
pip install datasets pandas scikit-learn pyyaml

# 3. Data preparation
python scripts/preprocess_data.py \
    --train_samples_per_class 40 \
```

```
    --test_samples_per_class 10 \
    --output_dir sample_data \
    --seed 42

# 4. Fine-tuning
python scripts/train.py --config configs/train.yaml

# 5. Standalone inference (single message)
python scripts/inference.py \
    --config configs/inference.yaml \
    --message "I need to change my PIN number"

# 6. Full evaluation
python scripts/inference.py --config configs/inference.yaml --evaluate
```

Listing 8: Local reproduction commands (Linux / WSL).

9.3 Determinism Notes

All random sources are seeded:

- Data sampling: `seed=42` (in `preprocess_data.py`).
- Train/eval shuffling: same seed.
- Trainer: `seed=42`, `lora.random_state=3407`.
- Inference: greedy decoding (`do_sample=false`, `temperature=0`).

However, exact reproduction of the **87.92%** headline number can vary slightly run-to-run because Unsloth, BitsAndBytes, and CUDA kernels introduce small non-determinism in 4-bit quantization. We have observed a stable range of **87–89%** test accuracy across multiple H100 runs with the same hyperparameters, which is the expected variance for this setup.

10 Conclusion

We successfully fine-tuned the **LLaMA 3.2 3B Instruct** model with **QLoRA** via **Unsloth** on a sampled subset of the BANKING77 dataset, reaching **87.92%** test accuracy on a 77-class intent classification task in only **4.1 minutes** of training on a single H100 GPU. The deliverables – a public GitHub repository with the required folder structure, an autopilot Colab notebook with all outputs preserved, a standalone `IntentClassification` class with the required `__init__` / `__call__` signature, and a public demo video – collectively satisfy every item of the project rubric.

References

[1] Casanueva, I., Temcinas, T., Gerber, D., Fitzpatrick, M., & Vulic, I. (2020). *Efficient Intent Detection with Dual Sentence Encoders*. Proceedings of the 2nd Workshop on NLP for ConvAI – ACL 2020. https://arxiv.org/abs/2003.04807

[2] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685. https://arxiv.org/abs/2106.09685

[3] Unsloth AI. (2024). *Unsloth – Fine-tune LLMs 2-5x faster with 80% less memory*. https://github.com/unslothai/unsloth

[4] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized Language Models*. arXiv preprint arXiv:2305.14314. https://arxiv.org/abs/2305.14314

[5] Meta AI. (2024). *Llama 3 Model Card*. https://github.com/meta-llama/llama3

[6] von Werra, L., et al. *TRL – Transformer Reinforcement Learning*. https://github.com/huggingface/trl